DeepLabCut Tutorial

Pose Estimation for Behavioral Neuroscience

sirocampus · HPC Workflow Guide

Saved from sirocampus · March 2026

CHAPTER 00

Getting Started

Getting Started¶

This tutorial assumes you will run the DeepLabCut GUI inside a ThinLinc desktop session on the HPC.

Bundled DeepLabCut welcome image from the local `code/lib/DeepLabCut` checkout.

1. Start a ThinLinc session¶

Open ThinLinc and connect with:

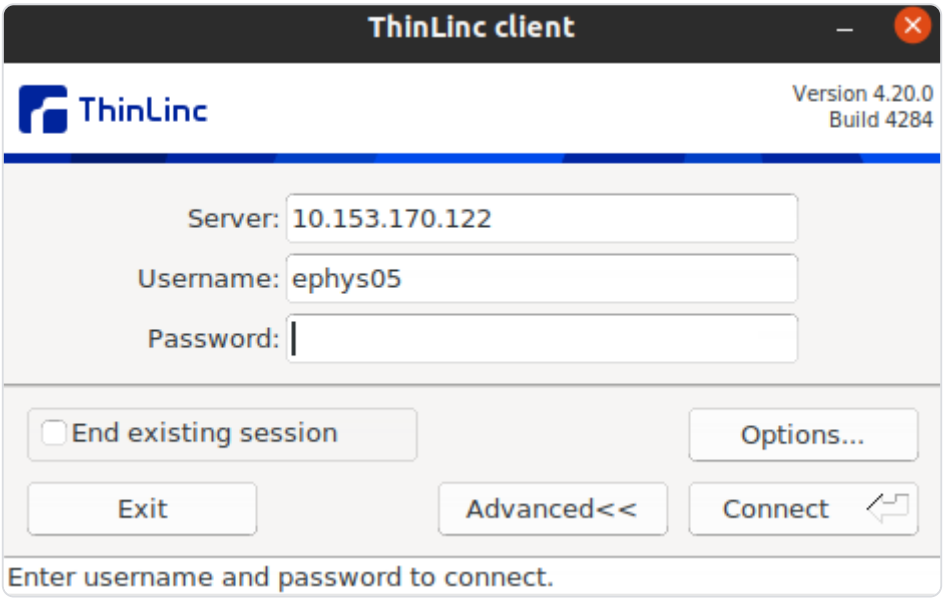
- Host: `10.153.170.122`
- Username: `ephysXX` where `XX` is `01` to `10`
- Password: ask your instructor

If you do not have the ThinLinc client installed yet, download it from:

- https://www.cendio.com/thinlinc/download/

After logging in, open a terminal inside the ThinLinc desktop.

ThinLinc login screen



This is the ThinLinc login window. After connecting successfully, continue inside the graphical ThinLinc session and open a terminal before launching DeepLabCut.

2. Activate the environments you need¶

Use the `labpy` environment for tutorial setup and DataLad commands:

```
conda activate labpy
```

Install the tutorial dataset into a parent directory of your choice. The command below uses `~/Desktop` as an example, but any writable parent directory works:

```
tutorial-deeplabcut ~/Desktop
cd ~/Desktop/deeplabcut-tutorial
```

This installs the dataset and retrieves `raw/` for you.

Why use DataLad here?
The tutorial repo stays lightweight at first. Retrieve only the data you need with `datalad get`.

Manual DataLad alternative
If you prefer to see the explicit setup steps, run:

```
mkdir -p ~/Desktop
cd ~/Desktop
git config --global --add safe.directory /storage/share/git/ria-store/e3d/53f54-c3a7-4790-8c4b-3b75eea1c
datalad install -s ria+file:///storage/share/git/ria-store#~tutorial-deeplabcut deeplabcut-tutorial
cd deeplabcut-tutorial
datalad get raw
```

Choosing the target folder
`tutorial-deeplabcut` takes a parent directory, not a fixed location. For example:

- `tutorial-deeplabcut ~/Desktop`
- `tutorial-deeplabcut ~/work/tutorials`
- `tutorial-deeplabcut .`

Tutorial repo layout
In this tutorial repo:

- `raw/` stores the input video.
- `dlc-tutorial-<INITIALS>-<DATE>/` will be created by DeepLabCut at the repo root when you make your project.
- `expected/` stores the completed reference state prepared by the instructor.

Then switch to the DeepLabCut environment for the GUI work:

Use the shared DeepLabCut environment on the HPC:

```
conda activate DEEPLABCUT
```

Alternative environment command
Some systems also provide a shared conda helper:

```
conda-shared
conda activate DEEPLABCUT
```

If you later need another DataLad command, switch back to `labpy` in that terminal or open a second terminal for setup/data work.

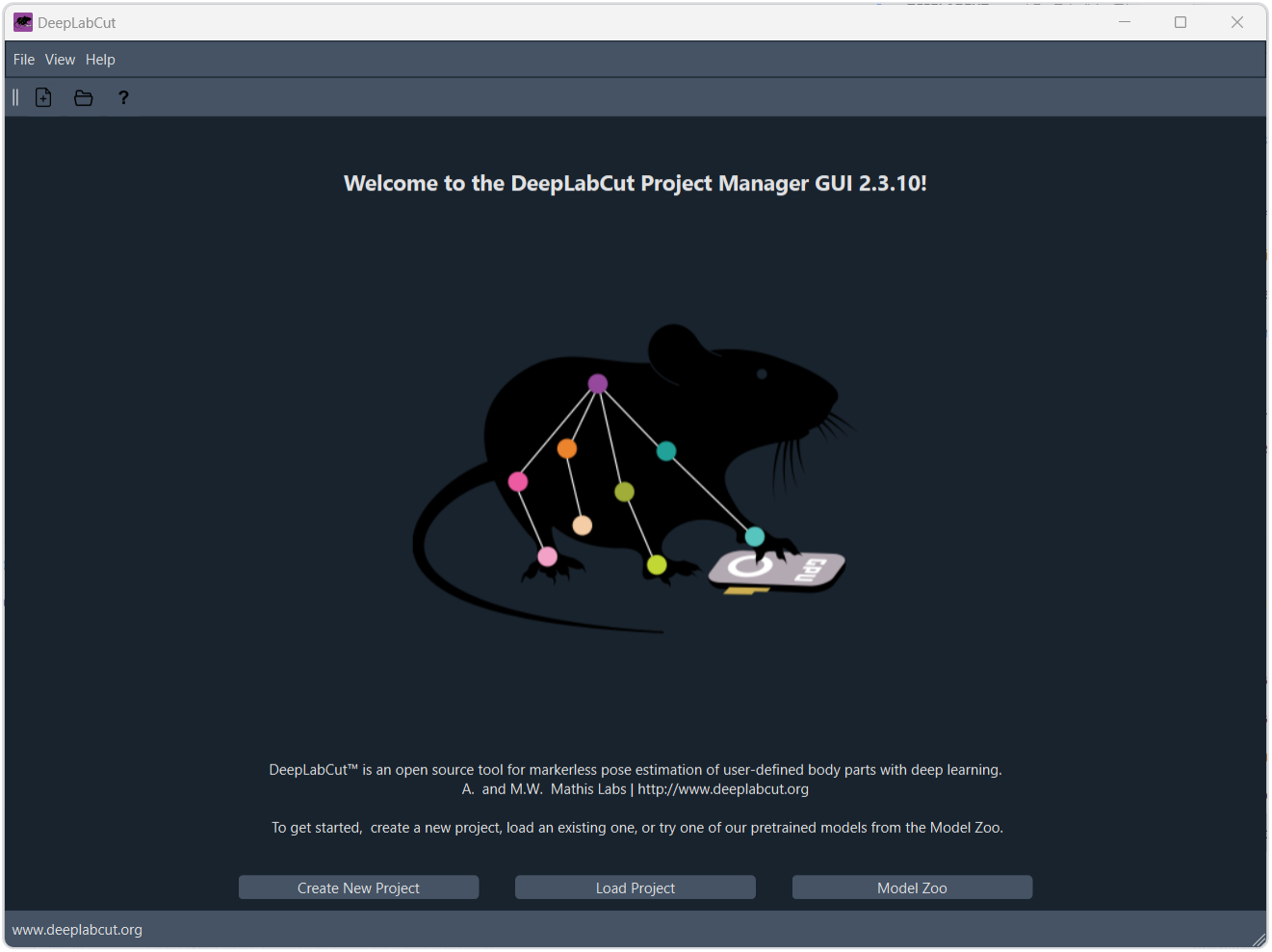
3. Launch the GUI¶

```
deeplabcut-gui
```

> **TIP**
>
> If the GUI does not appear, first confirm you are inside ThinLinc rather than a plain SSH session.

If the GUI does not open
If the GUI still does not open:

- Open a fresh terminal inside ThinLinc.
- Reactivate the environment.
- Launch `deeplabcut-gui` again.



DeepLabCut GUI home screen.

Back to: DeepLabCut tutorial contents
Next: Create a project and add data

CHAPTER 01

Project and Data

Create A Project And Add Data¶

Goal: create a new DeepLabCut project at the repo root and use the raw example video directly from `raw/`.

> **WARNING**
>
> Use the raw `.avi` as input, not the completed reference output in `expected/`.

1. Confirm the raw example video is present¶

If you used `tutorial-deeplabcut`, the installer already retrieved `raw/`. Confirm the input video is present:

```
cd ~/Desktop/deeplabcut-tutorial
ls raw/R25-T009_M-LTM-E20_recall_2024-07-23_13-44-19.Cam12.ds.cropped.avi
```

> **NOTE**
>
> These examples assume you installed the dataset into `~/Desktop`. If you chose a different parent directory, replace `~/Desktop/deeplabcut-tutorial` with your own path.

> **NOTE**
>
> If the file is missing, switch to the `labpy` environment and run `datalad get raw`.

2. Prepare the project location¶

For this tutorial, work inside the cloned repo itself. DeepLabCut will create a new project folder at the repo root, for example `dlc-tutorial-AS-2026-03-09`.

```
cd ~/Desktop/deeplabcut-tutorial
```

Raw input vs expected output

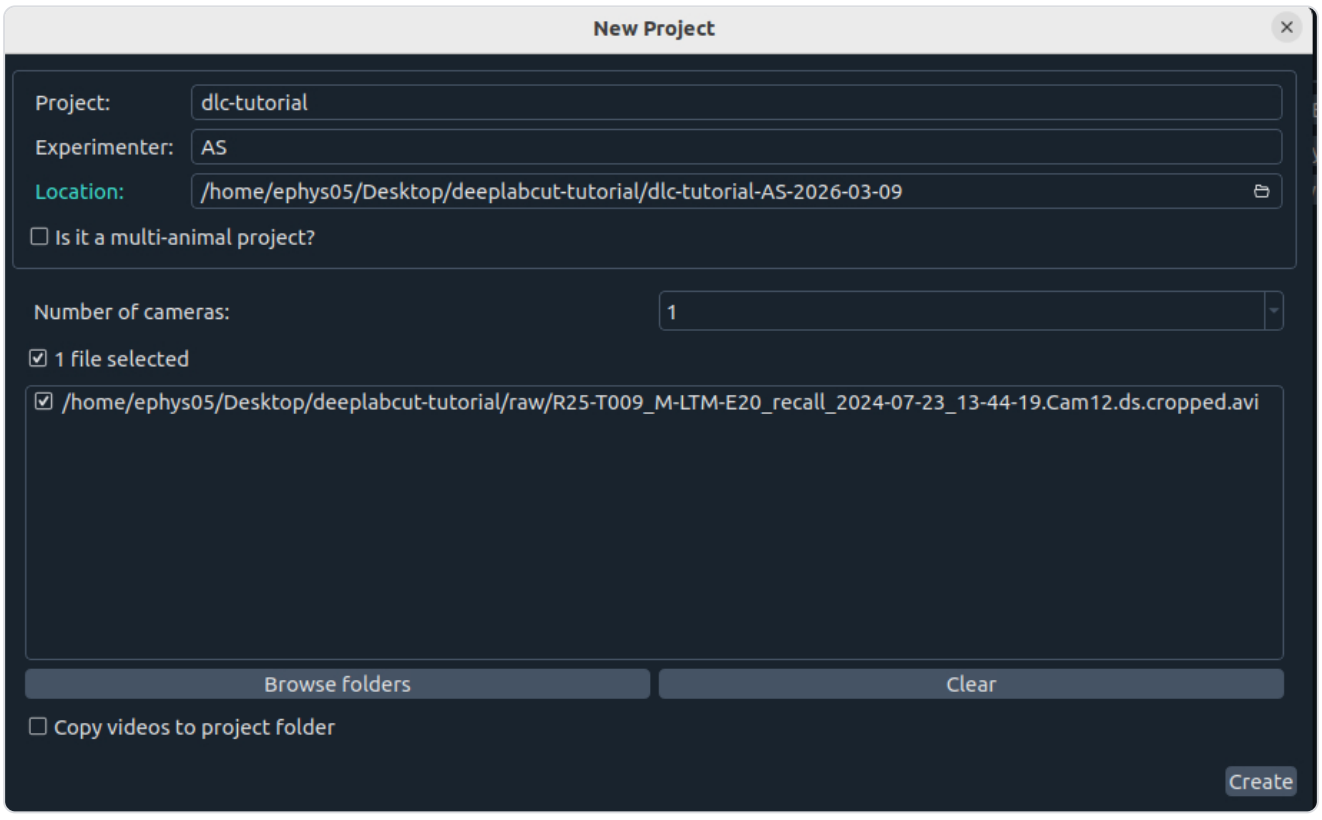
This repo contains both:

- Raw input video: `raw/R25-T009_M-LTM-E20_recall_2024-07-23_13-44-19.Cam12.ds.cropped.avi`
- Completed reference output: `expected/R25-T009_M-LTM-E20_recall_2024-07-23_13-44-19.Cam12.ds.croppedDLC_Resnet50_motive-practicumOct31shuffle1_snapshot_050_filtered_p60_labeled.mp4`

3. Create a new project in the GUI¶

In the DeepLabCut GUI:

1. Click `Create New Project`.
2. Set the project name to `dlc-tutorial`.
3. Enter your name or initials as the scorer.
4. In `Location`, choose `~/Desktop/deeplabcut-tutorial`.
5. Use `Browse folders` to select `raw/R25-T009_M-LTM-E20_recall_2024-07-23_13-44-19.Cam12.ds.cropped.avi`.
6. Leave `Copy videos to project folder` unchecked.



Example project creation dialog showing the repo root as the location and the input video selected directly from `raw/`.

Interactive Python alternative

If you prefer not to use the GUI for this step, use `ipython` in the terminal. It is a better interactive interface than plain `python` because it supports easier copy-paste, history, and multiline editing.

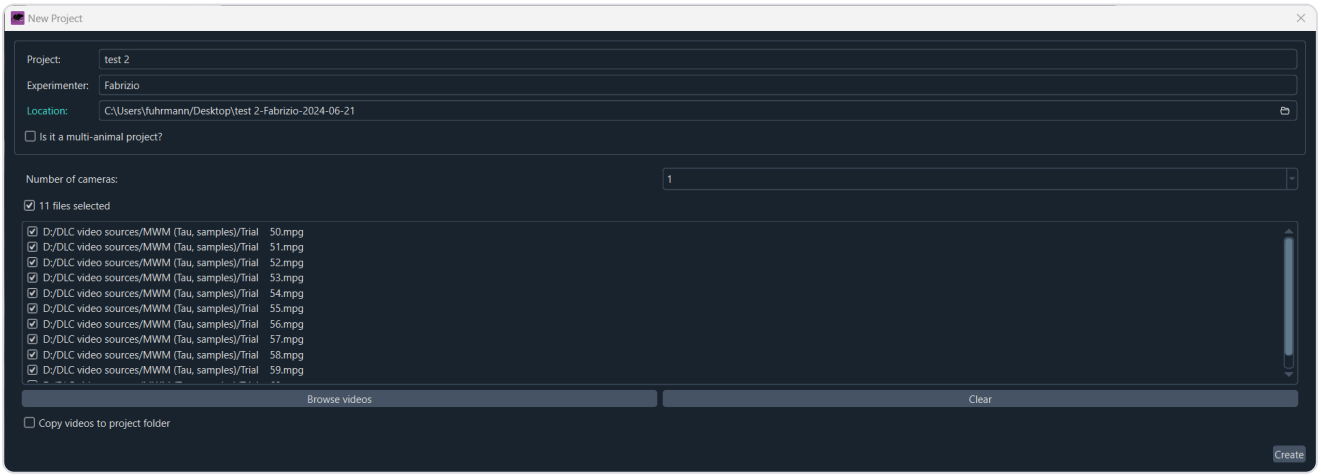
```
conda activate DEEPLABCUT
cd ~/Desktop/deeplabcut-tutorial
ipython
```

Then run:

```
import deeplabcut

deeplabcut.create_new_project(
    project="dlc-tutorial",
    experimenter="YOUR_INITIALS",
    videos=["raw/R25-T009_M-LTM-E20_recall_2024-07-23_13-44-19.Cam12.ds.cropped.avi"],
    working_directory=".",
    copy_videos=False,
    multianimal=False,
)
```

Replace `YOUR_INITIALS` with your own scorer name. This creates the same starter project at the repo root.



Create New Project dialog.

Supported video formats
The official GUI supports common video formats including `.mp4`, `.avi`, `.mkv`, and `.mov`. For this tutorial, stay with the provided `.avi` file so everyone uses the same input.

4. Verify the generated configuration¶

In a terminal with the `DEEPLABCUT` environment activated:

```
cd ~/Desktop/deeplabcut-tutorial/dlc-tutorial-*/
ls
```

You should see `config.yaml`.

Check that the configured video path points to the video in `raw/`:

```
python -c "from pathlib import Path; print(Path('config.yaml').resolve())"
python -c "import yaml; print(yaml.safe_load(open('config.yaml'))['video_sets'].keys())"
```

Interactive Python alternative for configuration checks
You can inspect the key config fields from `ipython` instead of clicking through the GUI:

```
conda activate DEEPLABCUT
cd ~/Desktop/deeplabcut-tutorial/dlc-tutorial-*/
ipython
```

Then run:

```
import yaml

config = yaml.safe_load(open("config.yaml"))
print("project_path:", config["project_path"])
print("video_sets:", list(config["video_sets"].keys()))
print("bodyparts:", config.get("bodyparts"))
print("skeleton:", config.get("skeleton"))
```

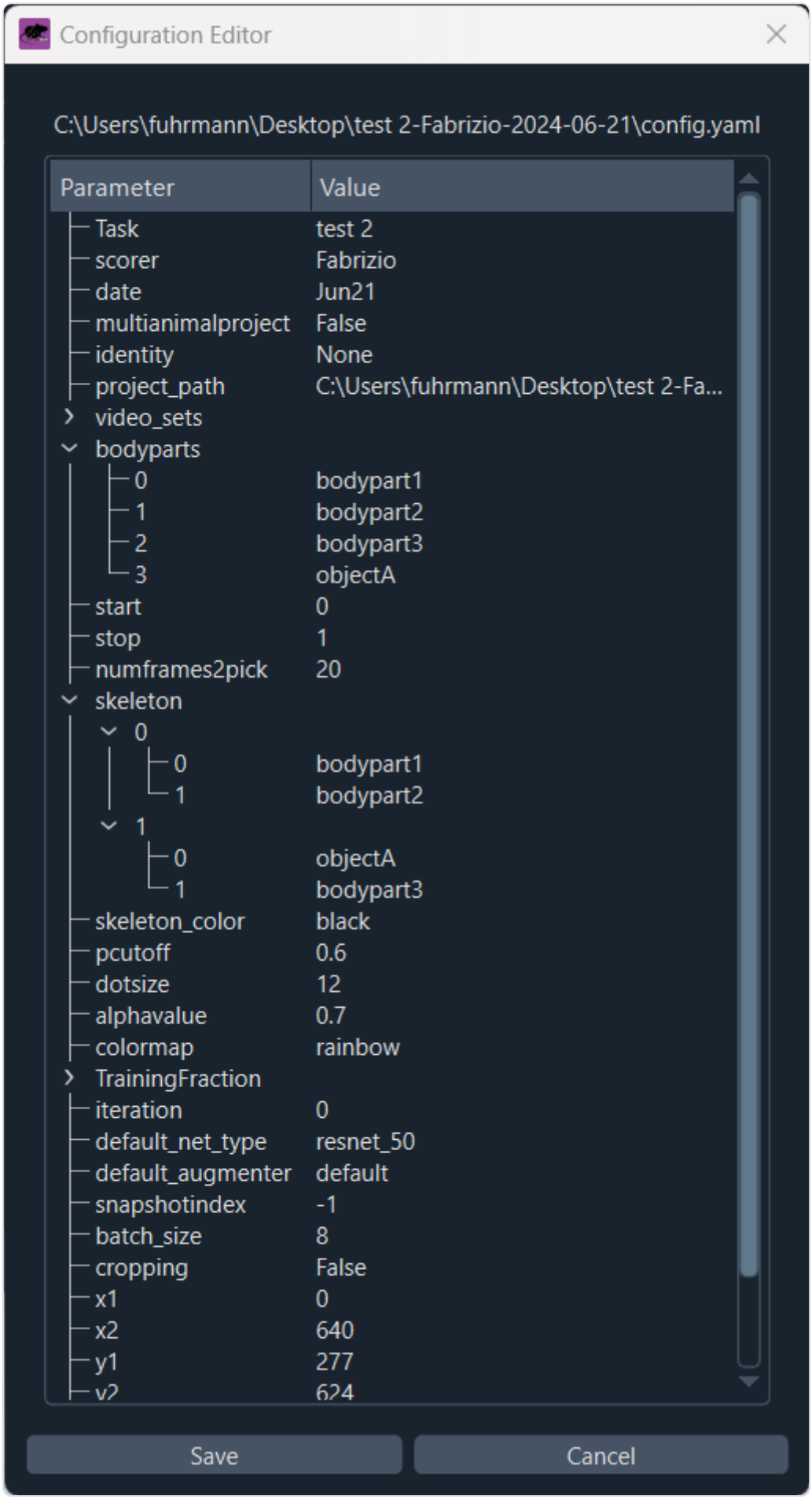
> **WARNING**
>
> If `video_sets` points to someone else's absolute path, fix it before continuing. This is a common failure mode after moving a DLC project.

5. Edit `config.yaml` from the GUI¶

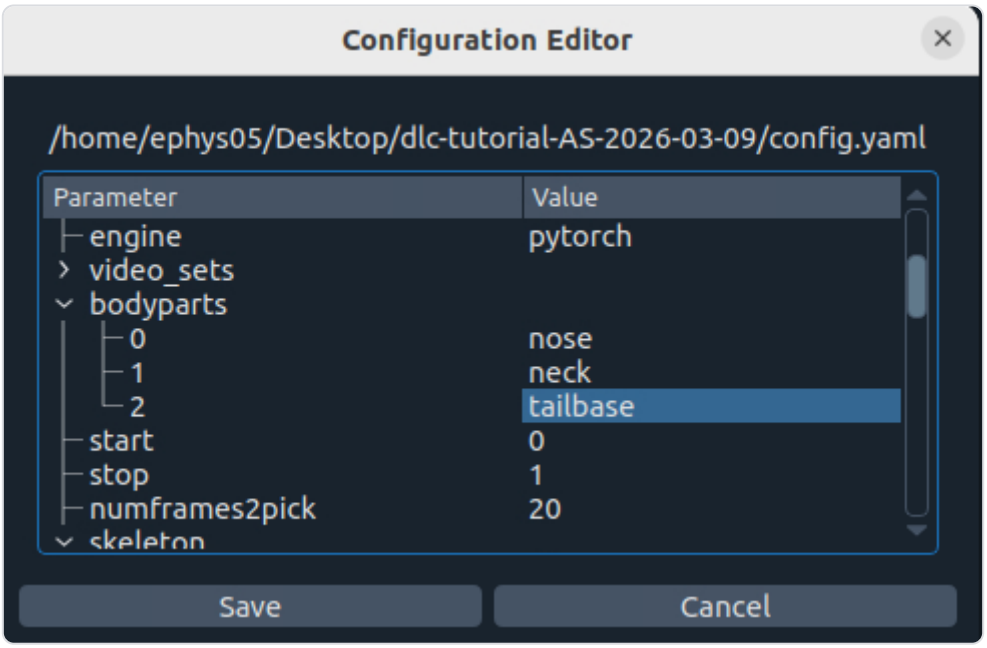
After project creation, go to the `Manage project` tab and click `Edit config.yaml`.

For this tutorial, make sure:

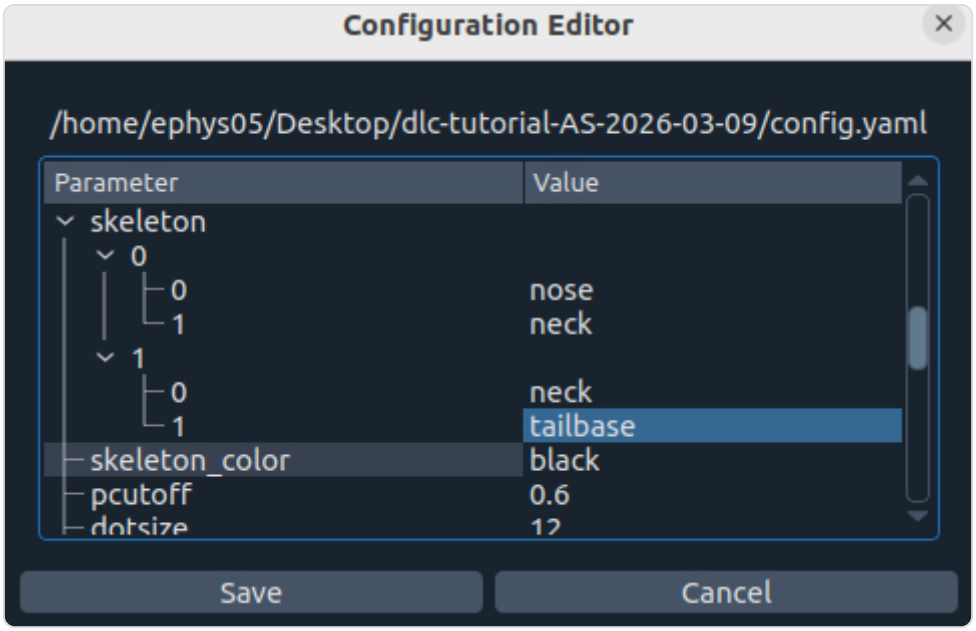
- `bodyparts` contains `nose`, `neck`, `tailbase`
- `skeleton` connects `neck` to `nose` and `tailbase`


Configuration Editor
C:\Users\fuhrmann\Desktop\test 2-Fabrizio-2024-06-21\config.yaml
Parameter
Value
Task
test 2
scorer
Fabrizio
date
Jun21
multianimalproject
False
identity
None
project_path
C:\Users\fuhrmann\Desktop\test 2-Fa...
video_sets
bodyparts
0
bodypart1
1
bodypart2
2
bodypart3
3
objectA
start
0
stop
1
numframes2pick
20
skeleton
0
0
bodypart1
1
bodypart2
1
0
objectA
1
bodypart3
skeleton_color
black
pcutoff
0.6
dotsize
12
alphavalue
0.7
colormap
rainbow
TrainingFraction
iteration
0
default_net_type
resnet_50
default_augmenter
default
snapshotindex
-1
batch_size
8
cropping
False
x1
0
x2
640
y1
277
y2
624
Save
Cancel

Bodyparts:



Skeleton:



Editing the project configuration file in the GUI.

BOX 1: Glossary of parameters in the project configuration file (config.yaml)

The config.yaml file sets the various parameters for generation of the training set file and evaluation of results. The meaning of these parameters is defined here, as well as referenced in the relevant step.

Parameters set during the project creation:

task: Name of the project (e.g. mouse-reaching). (do not edit) scorer: Name of the experimenter (do not edit)

date: Date of creation of the project. (do not edit)

project_path: Full path of the project; edit this if you need to move the project to a cluster/server/another computer or a different directory on your computer

video_sets: A dictionary with the keys as the full path of the video file and the values, crop as the cropping parameters used during frame extraction.
(use the function add_new_videos to add more videos to the project; if necessary the paths can be edited manually, and the crop values are designed to be edited manually).

engine: Default DeepLabCut engine to use for shuffle creation (either pytorch or tensorflow)

Important parameters to edit after project creation:

bodyparts: List containing names of the points to be tracked. Do not change after labeling frames (and saving labels).
You can add additional labels later, if needed.

numframes2pick: This is an integer that specifies the number of frames to be extracted from a video or a segment of video. The default is set to 20.

colormap: It specifies the colormap used for plotting the labels in images or videos in many steps.
Matplotlib colormaps are possible (https://matplotlib.org/examples/color/colormaps_reference.html).

dotsize: Specifies the marker size when plotting the labels in images or videos. The default is set to 12.

alphavalue: Specifies the transparency of the plotted labels. The default is set to 0.5.

iteration: This keeps the count of the number of iterations used to create the training dataset.
The first iteration starts with 0 and thus the default value is set to 0.
Do not change this manually.

If you are extracting frames from long videos (Value in relative terms of video length, i.e. [start=0,stop=1] is the full video)

start: Start point of interval to sample frames from when extracting frames. The default is set to 0.

stop: Same as start, but the end of the interval. Default is 1.

Related to the Neural Network Training:

TrainingFraction: This is a two digit floating-point number in the range [0-1] to split the dataset into training and testing dataset. The default is set to 0.95.

default_net_type: The default pose estimation model architecture to use when creating training datasets.

Used during video analysis:

batch_size: This specifies how many frames to process at once during inference (For tuning of this parameter see Mathis & Warren 2018).

snapshotindex: This specifies which checkpoint to use to evaluate the network. The default is set to -1. Use "all' to evaluate all the checkpoints.
Snapshots refer to the stored TensorFlow configuration, which holds the weights of the feature detectors

p-cutoff: This specifies the threshold of the likelihood and helps distinguishing likely body parts from uncertain ones. The default is set to 0.1.

cropping: Specifies if the analysis video needs to be cropped. The default is set to False.

x1,x2,y1,y2: These are the cropping parameters used for cropping novel videos). The default is set to the frame size of the video.

Used during refinement steps:

move2corner: In some (rare) cases the predictions from DeepLabCut will be outside of the image (due to the location refinement shifts).
This binary parameter makes sure that those points are mapped to a user defined point within the image so that the label can be manually moved to the correct location. The default is set to True.

corner2move2: This is the target location, if move2corner is True. The default is set to (50,50).

Reference figure from the local DeepLabCut docs showing the single-animal configuration file structure.

Terminal alternative for editing `config.yaml`

For editing, a text editor in the terminal is still simpler than interactive Python:

```
conda activate DEEPLABCUT
cd ~/Desktop/deeplabcut-tutorial/dlc-tutorial-*/
nano config.yaml
```

Set:

- `bodyparts` : `nose` , `neck` , `tailbase`
- `skeleton` : `[['neck', 'nose'], ['neck', 'tailbase']]`

Why keep the config minimal?
For this first walkthrough, keep the bodypart set intentionally small. The goal is to get one full run working end to end before adding more labels, more videos, or more complex settings. This also matches the official beginner guide, which treats `Edit config.yaml` as the main place to define bodyparts and the optional skeleton.

Project folder vs `expected/`
The DeepLabCut project folder you create at the repo root is your live working area as you follow the tutorial. `expected/` is the instructor-prepared completed reference state.

Background reading

- `bib/derivatives/docling/mathis_2018_DeepLabCutMarkerless/mathis_2018_DeepLabCutMarkerless.md`
- `bib/derivatives/docling/nath_2019_UsingDeepLabCut/nath_2019_UsingDeepLabCut.md`

Previous: Getting started
Next: Extract frames

CHAPTER 02

Extract Frames

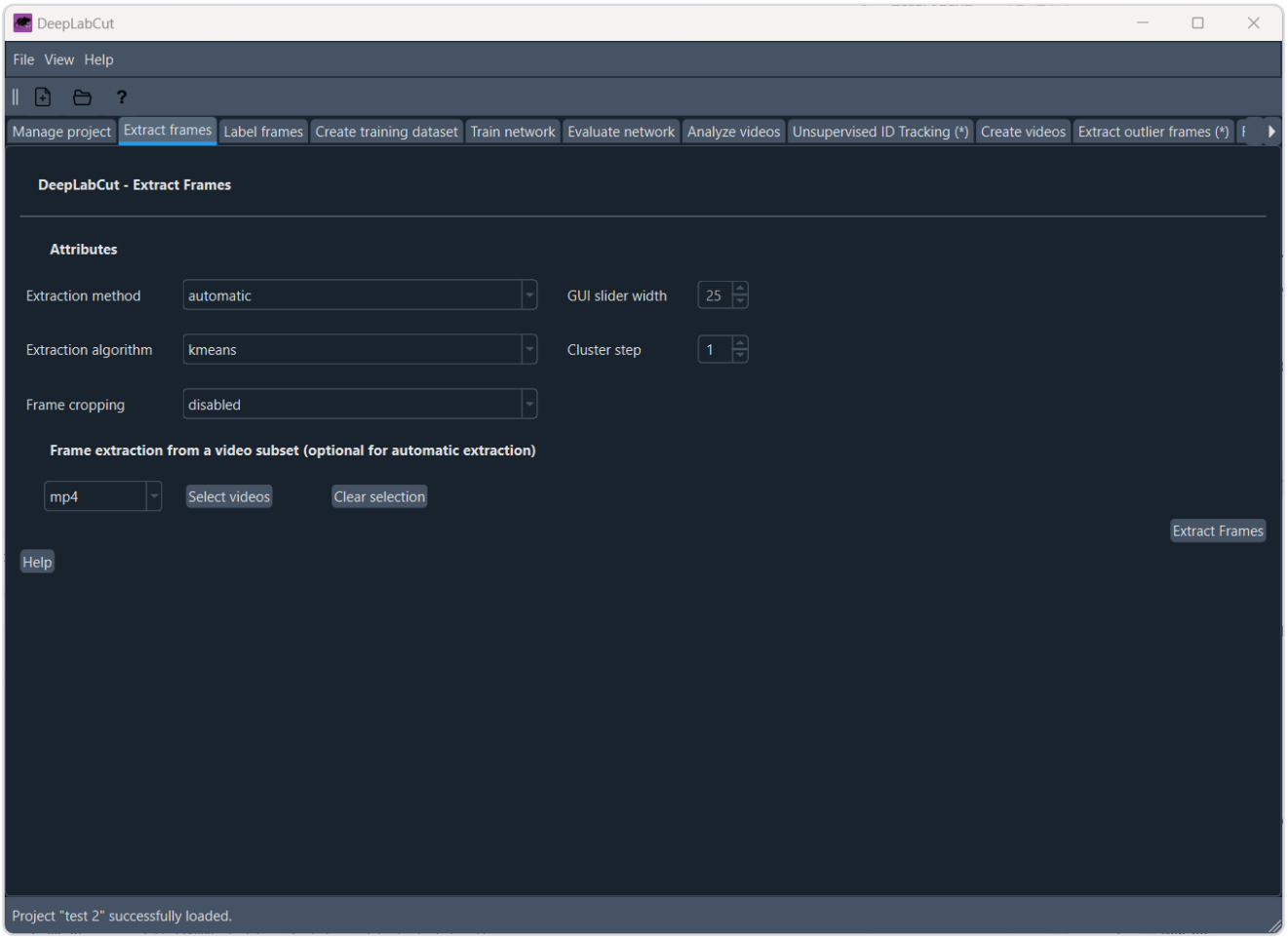
Extract Frames¶

Goal: extract a small but varied set of frames for manual labeling.

1. Try the GUI first¶

In the DeepLabCut GUI, open the `Extract frames` tab:

1. Choose `automatic`.
2. Use:
3. algorithm: `kmeans`
4. crop: `False`
5. Click `Extract Frames`.



> **WARNING**
>
> Change the video type to `avi` if your input video is `.avi`. The default is `mp4`, which can cause the frame extraction to fail silently without error messages. This is a common gotcha for new users.

Extract Frames panel.

2. Use the Python fallback if the GUI fails¶

> **WARNING**
>
> On remote HPC GUI sessions, `Extract Frames` can fail because of file picker issues, GUI hangs, or path handling problems.

If that happens, run the extraction step from Python instead.

Python fallback command
From the root of this tutorial repo:

```
conda activate DEEPLABCUT
python code/dlc_extract_frames.py --config /absolute/path/to/your/DLC/project/config.yaml
```

Example:

```
python code/dlc_extract_frames.py --config ~/Desktop/deeplabcut-tutorial/dlc-tutorial-*/config.yaml
```

This helper is intentionally non-interactive by default, so it extracts frames for all videos listed in `config.yaml` without prompting for confirmation.

3. Verify the extracted frames¶

DeepLabCut should create `labeled-data/<video-stem>/` inside your project.

```
cd ~/Desktop/deeplabcut-tutorial/dlc-tutorial-*/
ls -la labeled-data
```

> **TIP**
>
> Do not continue until you can see extracted images in the relevant `labeled-data` subdirectory.

How many frames get extracted?
By default, DeepLabCut uses `numframes2pick` from `config.yaml`. In many starter projects this is `20` frames per video, but the real source of truth is your config file.

Previous: Create a project and add data
Next: Label, train, and evaluate

CHAPTER 03

Label, Train, and Evaluate

Label, Train, And Evaluate¶

Goal: produce a first usable model from the extracted frames.

> **NOTE**
>
> This page keeps the pipeline intentionally short: label frames, create the training dataset, train once, then evaluate once.

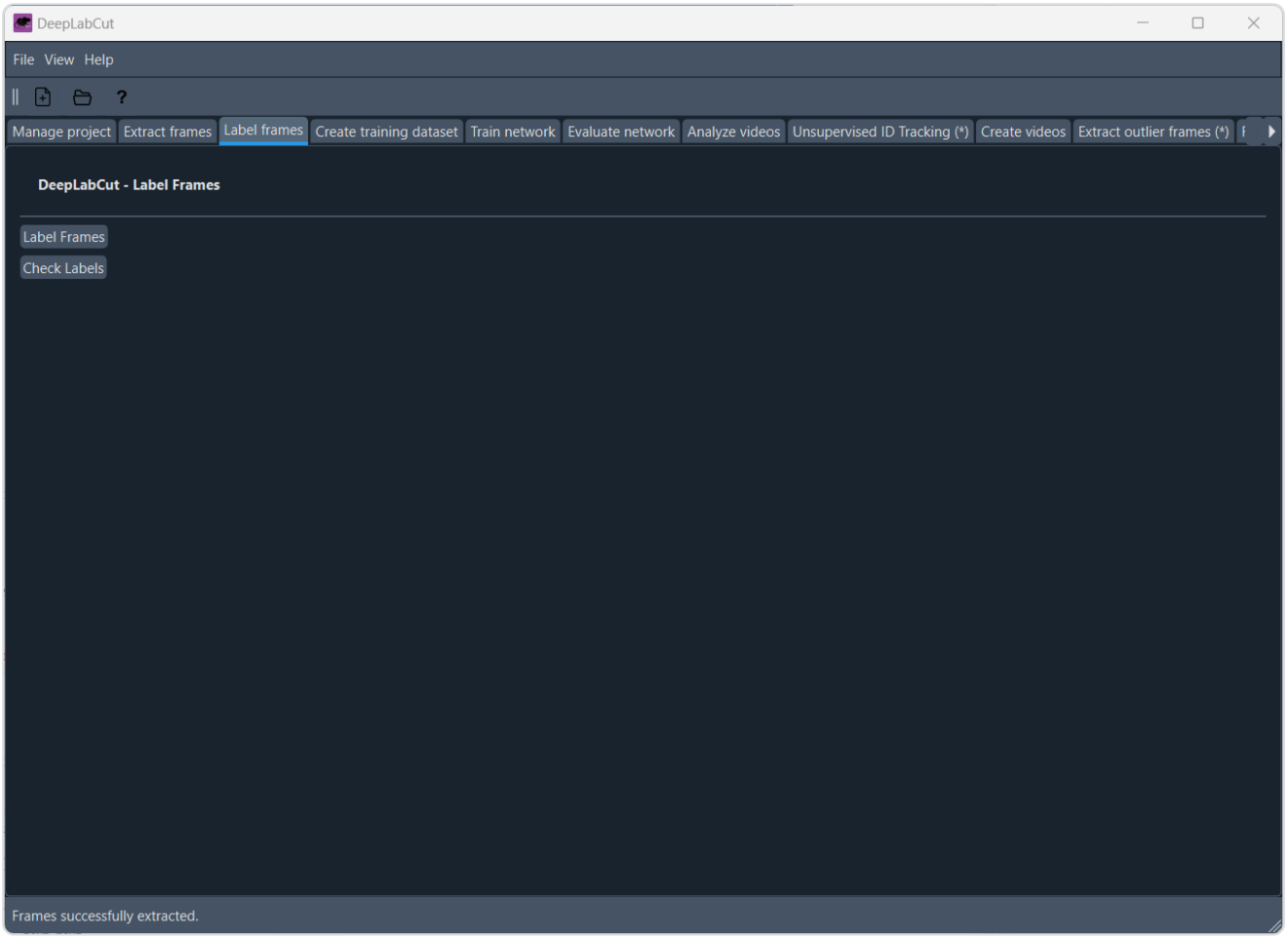
> **NOTE**
>
> Run these steps inside the DeepLabCut project folder you created at the repo root, for example `~/Desktop/deeplabcut-tutorial/dlc-tutorial-AS-2026-03-09`.

1. Label frames¶

In the GUI, open the `Label frames` tab:

1. Click `Label Frames`.
2. Open the folder under `labeled-data/` for your video.
3. Label `nose`, `neck`, and `tailbase` consistently across frames.

> **WARNING**
>
> The `napari` labeling window can take a minute to appear after you click `Label Frames`, especially on a shared HPC session. Wait for it to spawn before clicking repeatedly.



Label Frames window.

What napari is doing here
DeepLabCut opens the labeling interface in `napari`, an interactive image viewer used for stepping through frames and placing keypoints.

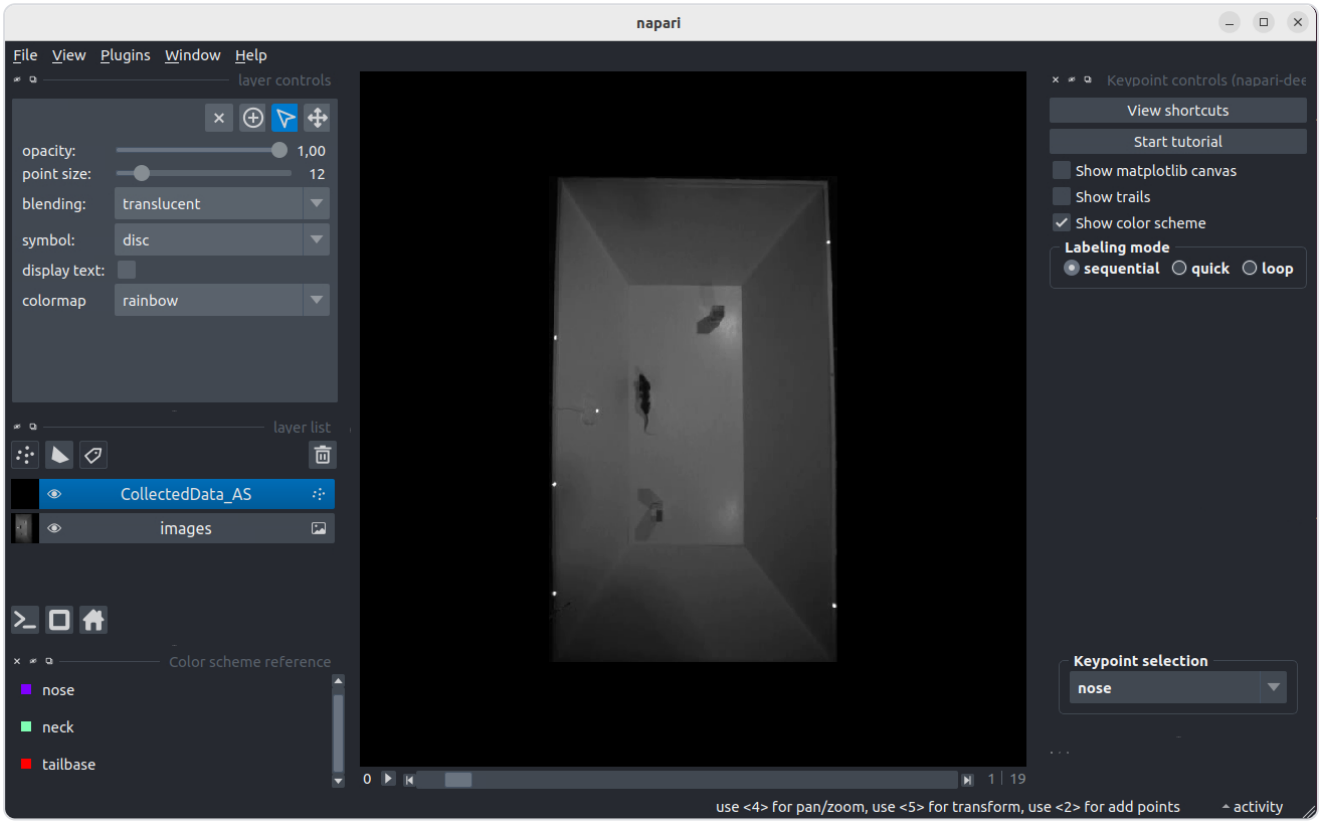
In practice:

- the image/frame is shown in the main viewer
- the bodyparts list appears as annotation controls
- you click on the image to place each keypoint
- you move frame by frame and save your labels as you go

If the window appears separate from the main DeepLabCut GUI, that is expected: the GUI launches `napari` as the labeling app.

Labeling rules
Two rules matter here:

- Use one consistent definition of `neck` across the whole dataset.
- If a point is genuinely not visible, do not guess its position.



Napari labeling interface.

Quick Napari tips

- Use the scroll wheel to move through frames.
- Click on the image to place a keypoint.
- Use the bodyparts list to select which keypoint you are placing.
- Save your labels frequently.

Where the labels are stored

DeepLabCut writes the frame images and labels under `labeled-data/` inside your project folder. That is the first place to inspect if labeling appears incomplete or a later step cannot find annotated frames.

2. Check labels¶

Still in the `Label frames` tab, click `Check Labels` and inspect a few frames before training.

Check Labels output

`Check Labels` overlays your annotations on the extracted frames. At this stage, look for:

- swapped bodyparts
- points consistently shifted away from the animal
- frames where one label was forgotten entirely

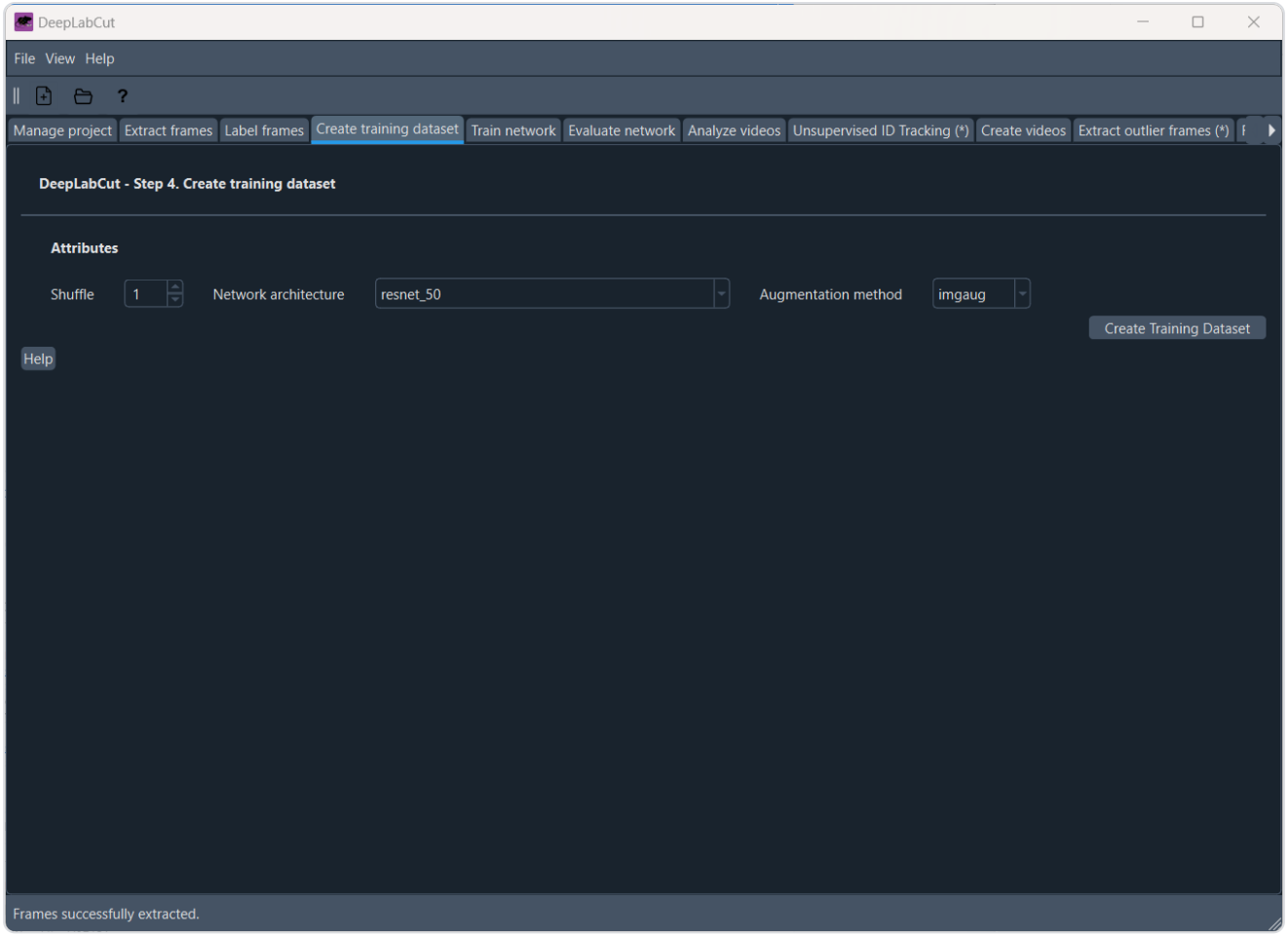
TIP

Fix obvious label mistakes now. Bad labels are harder to diagnose after training starts.

3. Create the training dataset¶

In the GUI, open the `Create training dataset` tab:

1. Click `Create Training Dataset`.
2. Keep the defaults for this first pass.

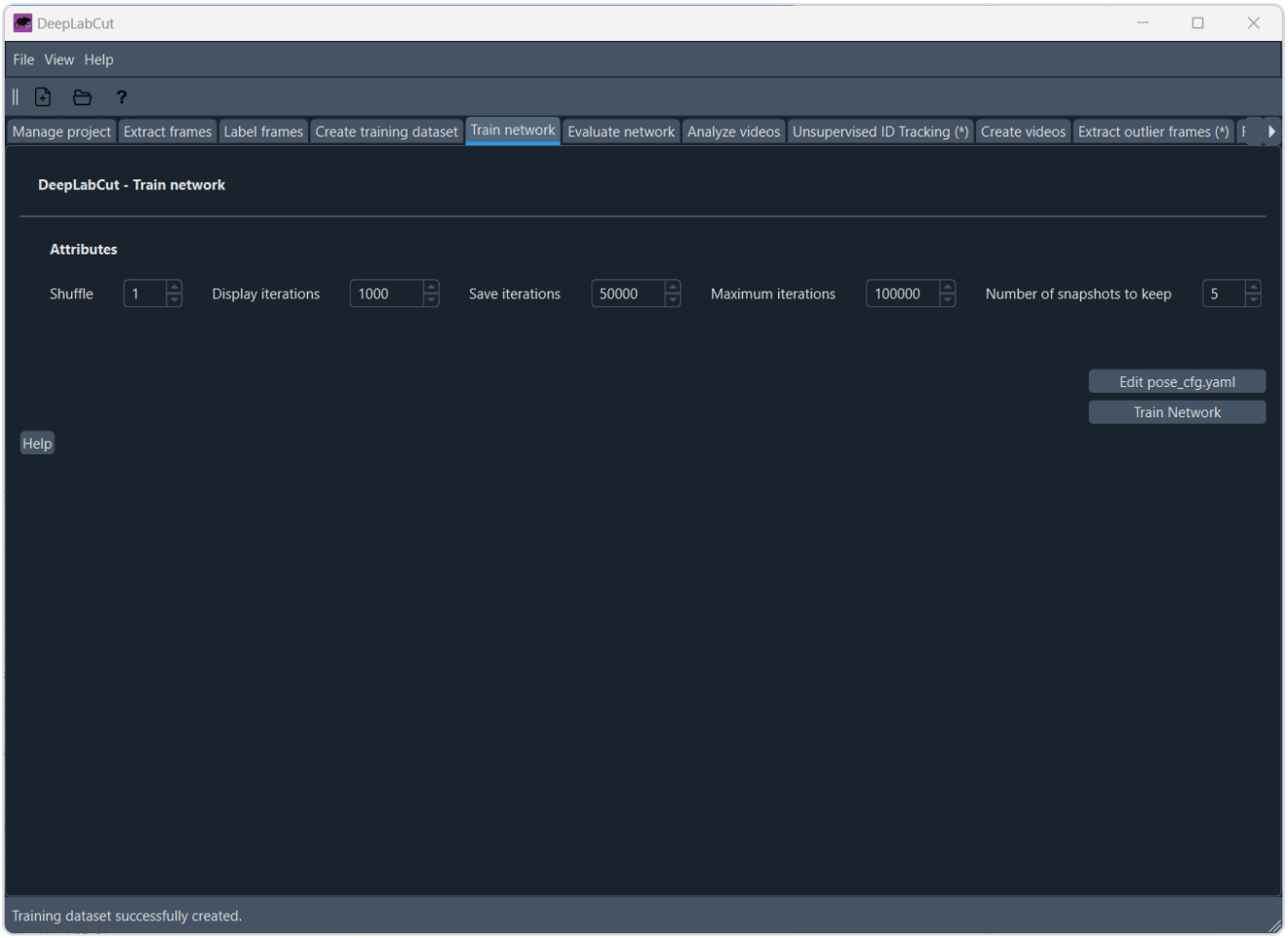


Create Training Dataset step.

4. Train the network¶

In the GUI, open the `Train network` tab:

1. Click `Train Network`.
2. Use a GPU if one is available.



Train Network tab.

CPU vs GPU
CPU-only training works, but it is much slower. The official beginner docs also note that progress is easiest to monitor from the terminal while training runs. For a classroom walkthrough, the goal is not a perfect model. The goal is to see the full workflow complete and produce reasonable predictions.

What to expect while training
Training can run for a while without much visible change in the GUI. That is normal. Watch the terminal for progress messages and wait for the run to finish cleanly before moving on.

5. Evaluate the network¶

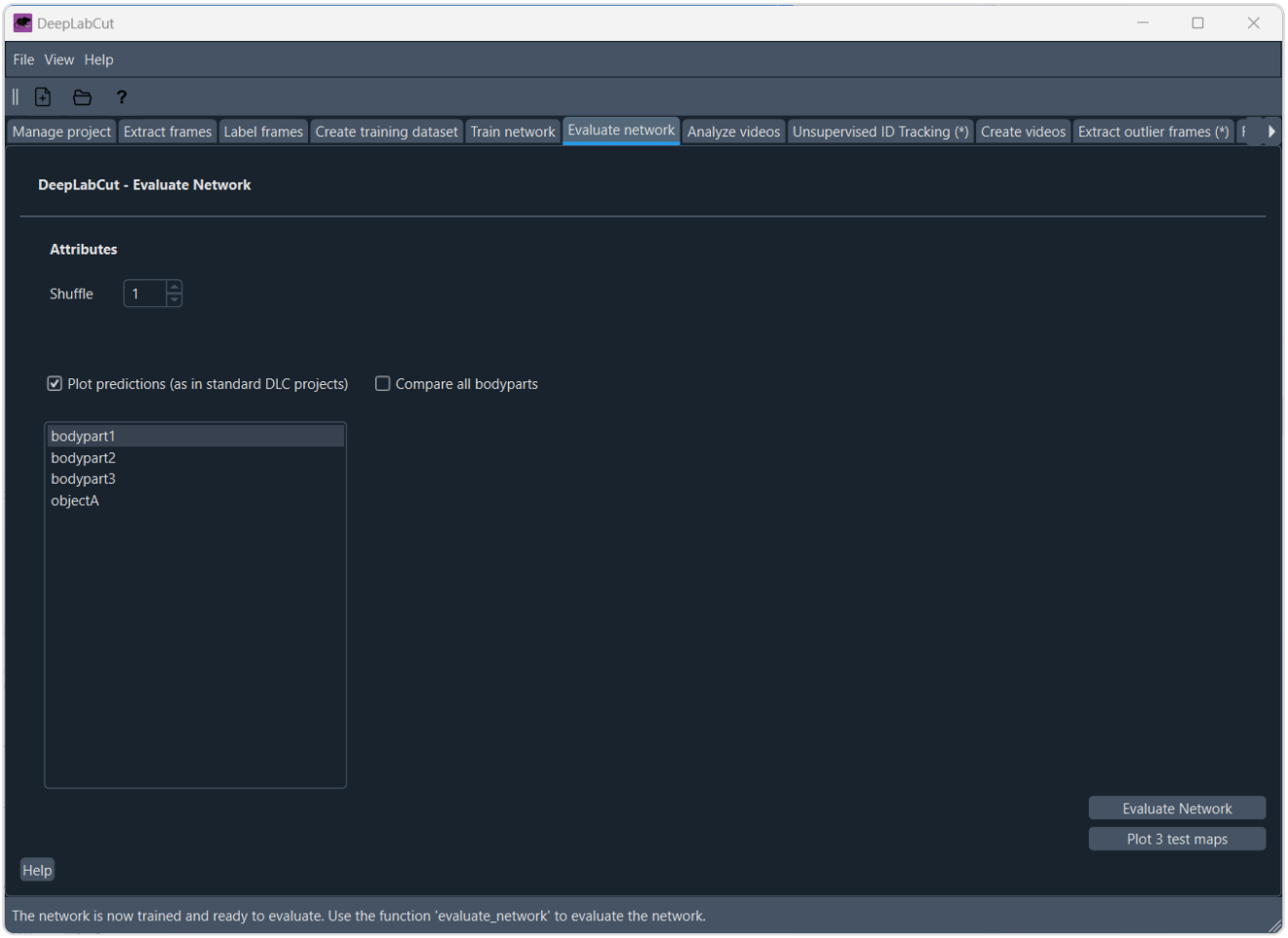
In the GUI, open the `Evaluate network` tab:

1. Click `Evaluate Network`.
2. If available, also inspect training history or evaluation plots.

Reasonable output for this stage means:

- the run completes without errors

- the error is not obviously extreme
- predicted points look plausible when visualized



Evaluate Network tab.

Where evaluation outputs go

For PyTorch projects, evaluation outputs are typically written under `evaluation-results-pytorch/`. The GUI may also create visual overlays that help you compare predictions against labels.

Background reading

- `bib/derivatives/docling/nath_2019_UsingDeepLabCut/nath_2019_UsingDeepLabCut.md`

Previous: Extract frames
Next: Analyze and export results

CHAPTER 04

Analyze and Export

Analyze And Export¶

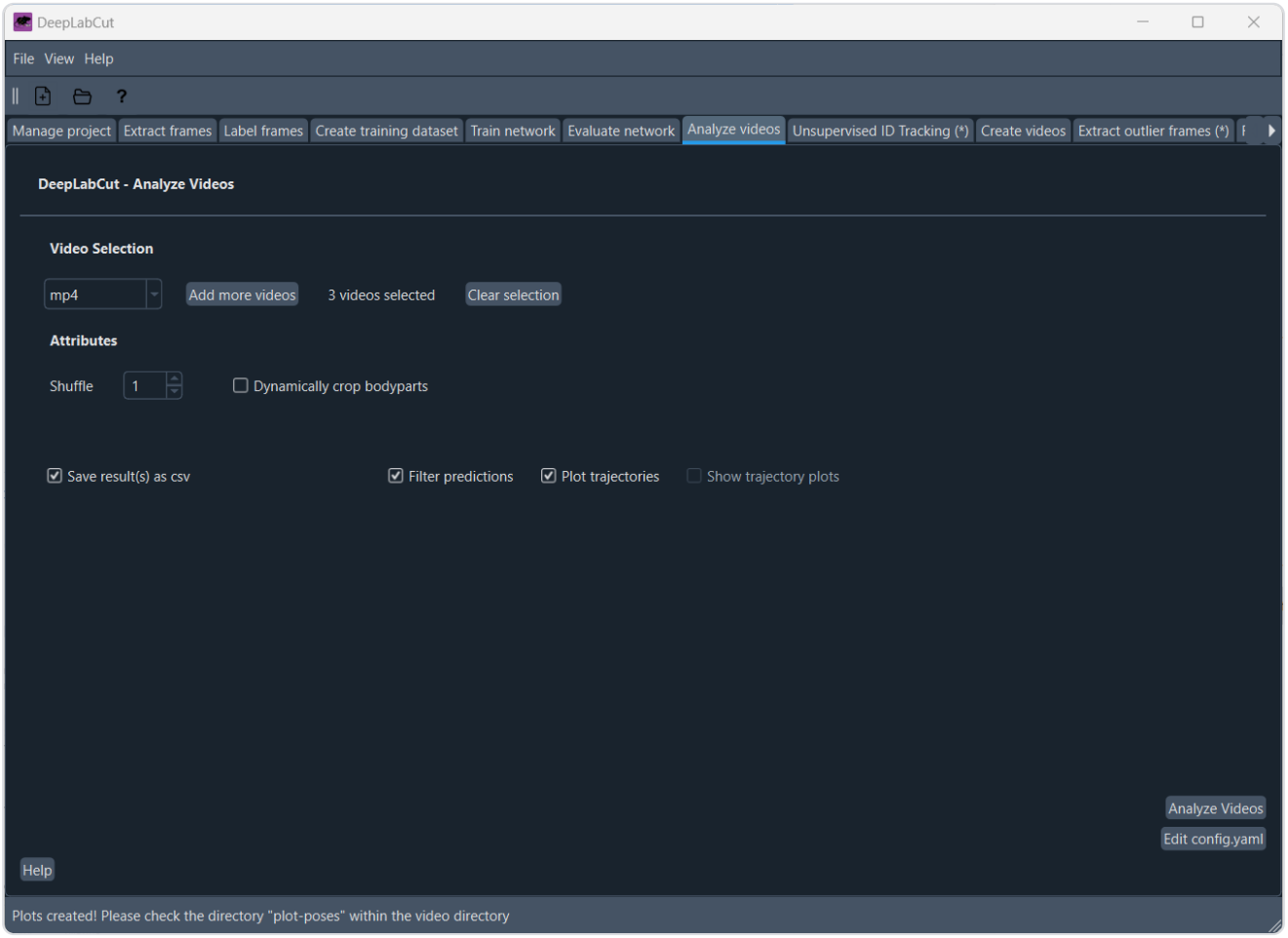
Goal: run inference on the raw video and produce a labeled output video.

1. Analyze the raw video¶

In the GUI, open the `Analyze videos` tab:

1. Select the raw `.avi` that belongs to your project.
2. Click `Analyze`.

This should generate prediction files such as `.h5`, `.csv`, and metadata files associated with the analyzed video.



Analyze Videos tab.

2. Optionally filter predictions¶

If your GUI version exposes filtering directly, run `Filter Predictions` and use the defaults for a first pass.

Filter Predictions step
Add a real `Filter Predictions` screenshot here later if your installed GUI exposes that control.

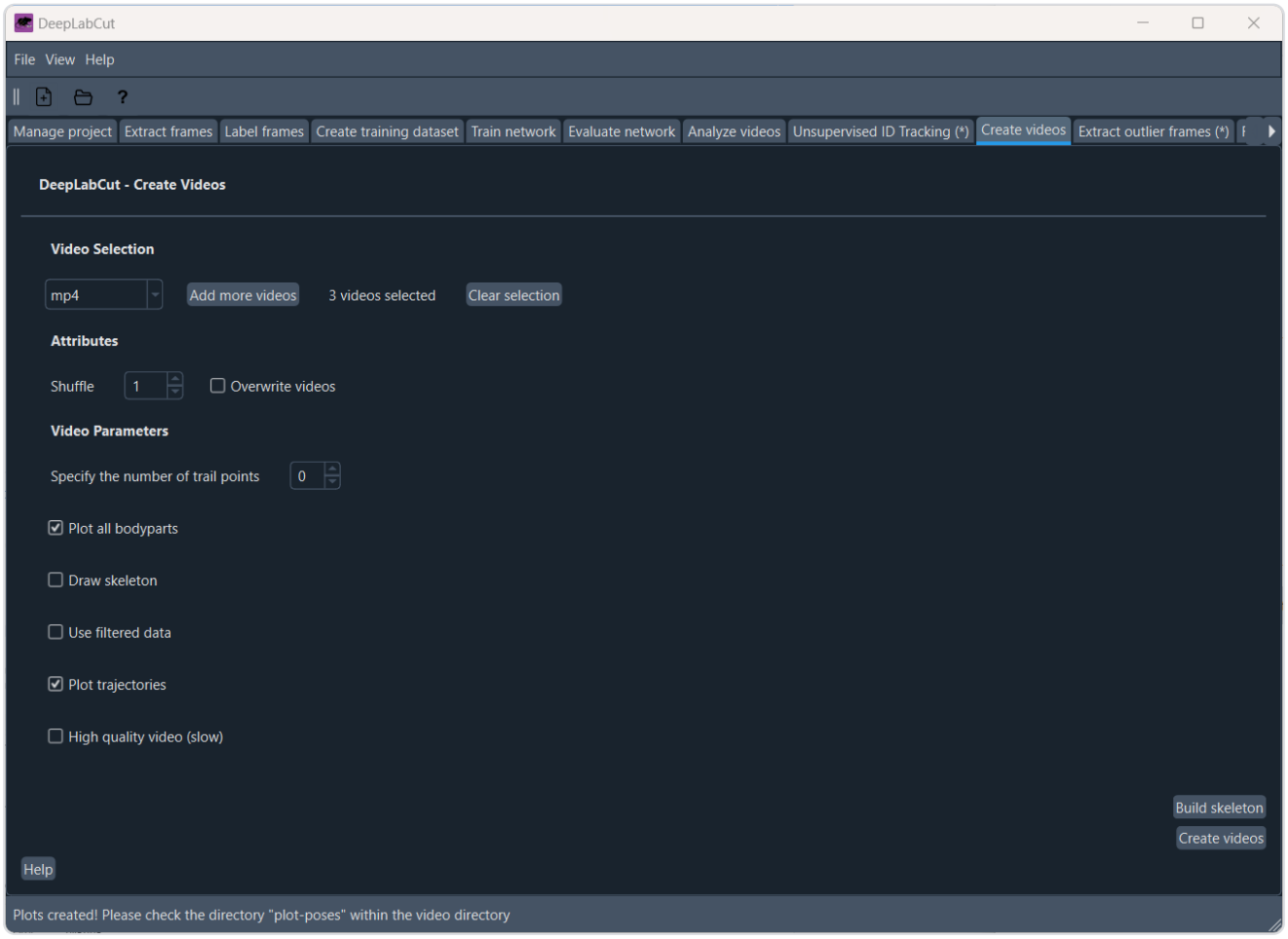
Why filter predictions?
Filtering is not mandatory for a first tutorial run, but it often reduces visible jitter and produces a cleaner labeled video.

3. Create the labeled video¶

In the GUI, open the `Create videos` tab:

1. Select the analyzed raw `.avi`.
2. Click `Create Videos`.

The result should be a rendered video with predicted keypoints overlaid.



Create Videos tab.

4. Copy the result into `expected/` ¶

From the root of this tutorial repo:

```
cp -v /absolute/path/to/your/project/videos/*labeled*.mp4 expected/
```

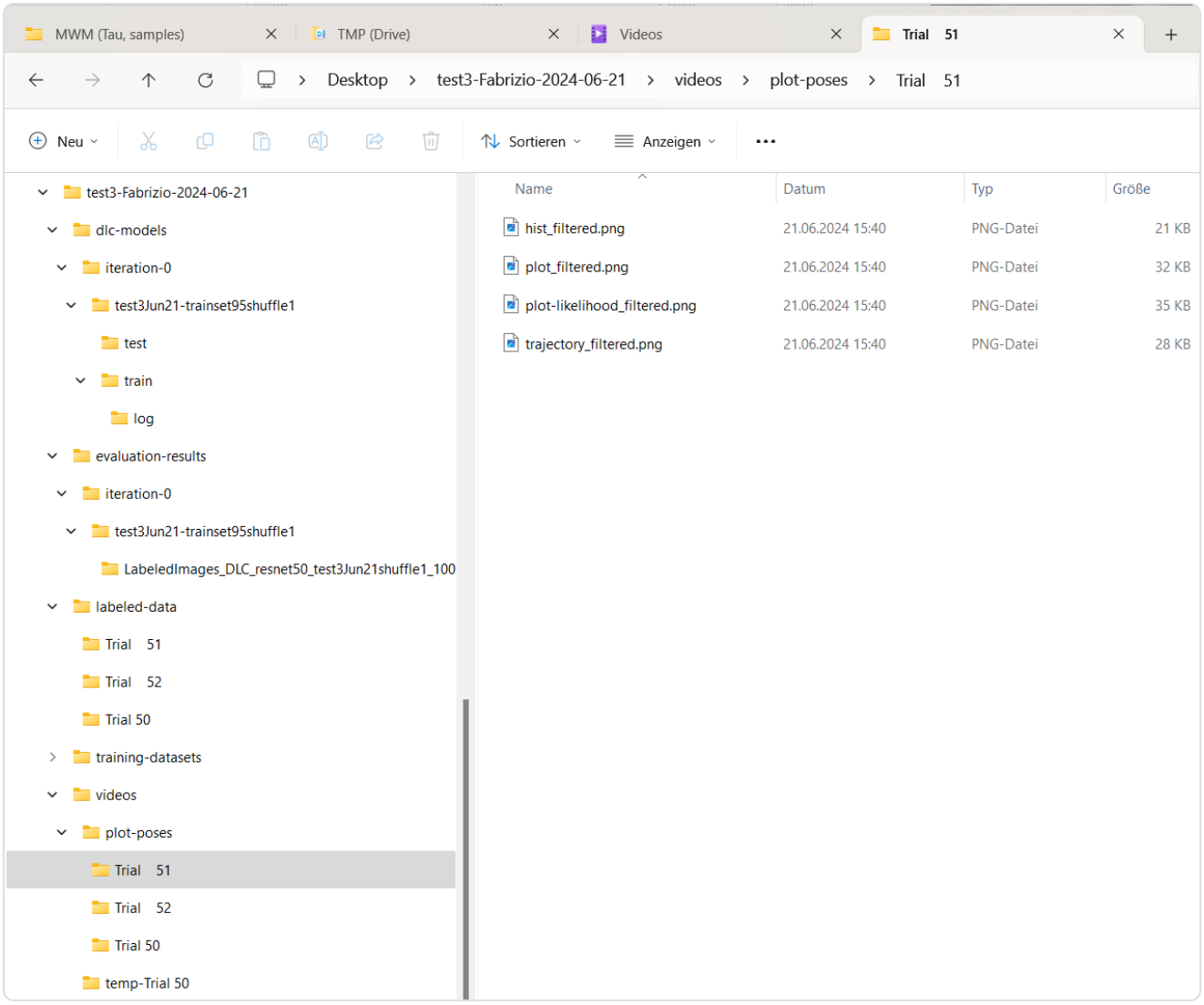
If your output is `.avi` instead of `.mp4`, adjust the command accordingly.

> **NOTE**
>
> Keep tutorial inputs in `raw/`, your live DeepLabCut project at the repo root, and instructor reference results in `expected/`.

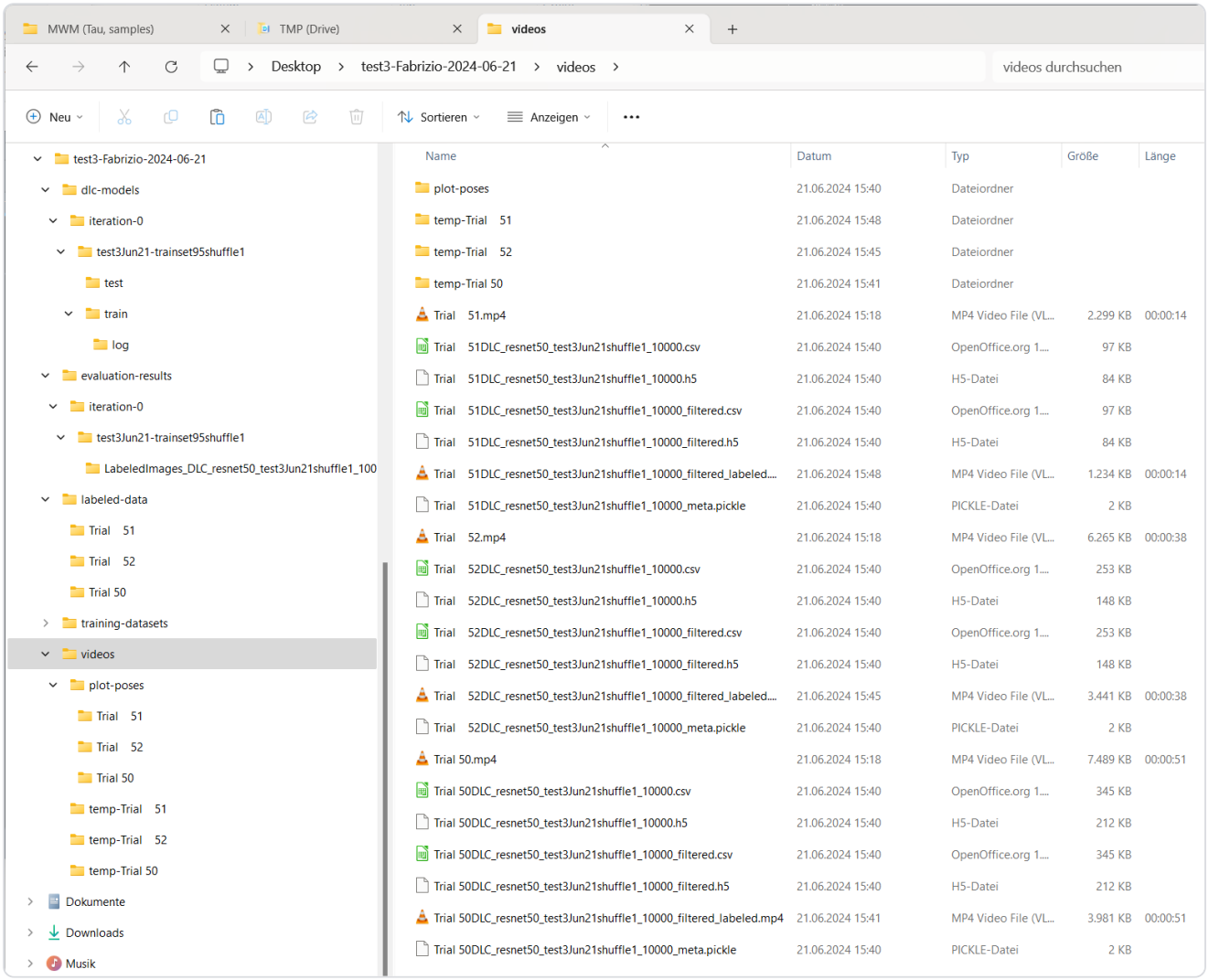
Example final output
This repo already includes one example output for reference:

`expected/R25-T009_M-LTM-E20_recall_2024-07-23_13-44-19.Cam12.ds.croppedDLC_Resnet50_motive-practicumOct31shuffle1_snapshot_050_filtered_p60_labeled.mp4`

How to inspect results
The official beginner guide also points students to the per-video output folder and the `plot-poses` subfolder when reviewing results. If your labeled video looks wrong, inspect those outputs before retraining.



Example `plot-poses` output folder.



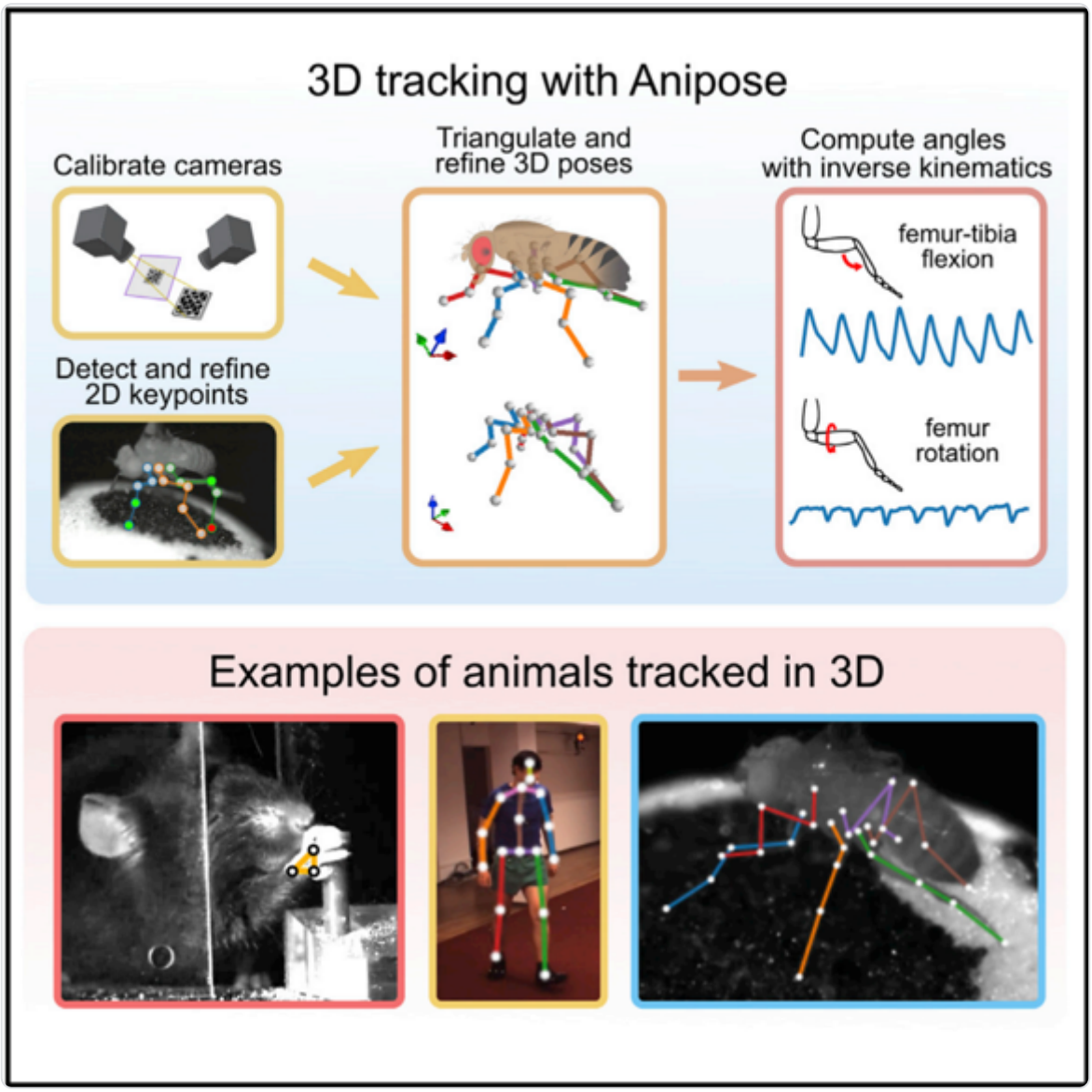
Example video output folder containing rendered results.

Variants and related tools¶

Beyond this tutorial

- Multi-animal DeepLabCut adds keypoint assembly, identity handling, and tracking for interacting animals.
- 3D pose estimation requires multiple synchronized cameras plus calibration and triangulation.
- Anipose is a practical 3D toolkit built around 2D pose estimates from tools such as DeepLabCut. It focuses on calibration, triangulation, filtering, and batch processing.

Multi-animal DeepLabCut figure from Lauer et al. (2022).



Anipose graphical abstract from Karashchuk et al. (2021), showing a practical 3D pipeline built on top of 2D pose estimates.

Background reading

- `bib/derivatives/docling/mathis_2018_DeepLabCutMarkerless/mathis_2018_DeepLabCutMarkerless.md`
- `bib/derivatives/docling/nath_2019_UsingDeepLabCut/nath_2019_UsingDeepLabCut.md`

Previous: Label, train, and evaluate
Back to: DeepLabCut tutorial contents